

## **Entrenamiento de modelo de aprendizaje supervisado**

### **Predicción de Churn - Caso Telmex Claro**

Gabriel David Rincón Bohórquez

Julian Esteban Ruiz Benavides

Orladis Muñoz Gómez

Universidad Piloto de Colombia

Aprendizaje de Máquina

Docente Cristian Camilo Tirado Cifuentes

12 de febrero de 2026

## Tabla de contenido

|                                                            |    |
|------------------------------------------------------------|----|
| Introducción .....                                         | 4  |
| Objetivo.....                                              | 4  |
| Contexto del problema .....                                | 4  |
| Datos etiquetados .....                                    | 5  |
| Definición y características .....                         | 5  |
| Variables del dataset .....                                | 5  |
| Generación de etiquetas con lógica de negocio .....        | 6  |
| Algoritmos de clasificación y regresión.....               | 8  |
| Comparación de algoritmos implementados .....              | 8  |
| Árbol de decisión .....                                    | 9  |
| Random forest .....                                        | 9  |
| Conjuntos de entrenamiento y prueba.....                   | 11 |
| Importancia de la partición.....                           | 11 |
| Estrategia de partición implementada .....                 | 11 |
| Preprocesamiento de datos.....                             | 11 |
| Validación cruzada .....                                   | 12 |
| Evaluación de rendimiento .....                            | 13 |
| Métricas de clasificación.....                             | 13 |
| Matriz de confusión .....                                  | 13 |
| Implicaciones de negocio.....                              | 14 |
| Curvas ROC y precision-recall .....                        | 15 |
| <b>Curva ROC (receiver operating characteristic)</b> ..... | 15 |
| <b>Curva recision-Reecall</b> .....                        | 16 |
| Importancia de variables .....                             | 17 |
| Generalización y ajustes del modelo.....                   | 19 |
| Análisis de overfitting/underfitting .....                 | 19 |
| Estrategias de regularización .....                        | 19 |
| Optimización de hiperparámetros.....                       | 20 |
| Conclusiones y recomendaciones .....                       | 21 |

|                                  |    |
|----------------------------------|----|
| Hallazgos principales.....       | 21 |
| Recomendaciones técnicas.....    | 21 |
| Recomendaciones de negocio ..... | 21 |
| Aporte en prospectiva .....      | 22 |
| Referencias .....                | 23 |

## Introducción

Este documento complementa el notebook de Python que desarrollamos para la Unidad 2.

La idea es profundizar en técnicas de entrenamiento supervisado, algo que en la industria de telecomunicaciones especialmente en los contact center es bastante crítico para predecir cuándo un cliente va a cancelar el servicio lo que llamamos churn.

### Objetivo

Queremos implementar y comparar tres algoritmos de clasificación: Regresión Logística, Árbol de Decisión y Random Forest, de esta manera poder ver cuál funciona mejor para predecir el churn en Telmex Claro que es el ejemplo que hemos utilizado, utilizamos herramientas de validación cruzada y optimizando los hiperparámetros para sacar el máximo provecho a cada modelo.

### Contexto del problema

El churn es uno de los problemas más costosos en telecomunicaciones y el cual año tras año afecta indicadores de cancelación, en el mundo de los contact center se han mostrado que:

- El costo de adquirir un nuevo cliente es 5-10 veces mayor que retener uno existente
- Una reducción del 5% en churn puede incrementar las utilidades entre 25% y 95%
- El 80% de los clientes que cancelan muestran señales predictivas en los 30-60 días previos
- Las acciones de retención proactivas tienen una tasa de éxito del 15-40%

## Datos etiquetados

Una de las cosas que diferencia el aprendizaje supervisado del no supervisado es que necesitamos datos etiquetados - es decir, ejemplos donde ya sabemos la respuesta. En nuestro caso, sabemos qué clientes cancelaron y cuáles no y de esta manera poder dar un ejercicio un poco más adaptado a la realidad.

### Definición y características

En este proyecto manejamos:

- Target (etiqueta o tag): churn\_30 - Indica si el cliente canceló en los próximos 30 días (1 = Sí, 0 = No).
- Features (variables predictoras): 11 características que describen el comportamiento del cliente.
- Tamaño del dataset: 10,000 clientes con información completa (Generado a través del código python).
- Período de análisis: Datos sintéticos con lógica de negocio del sector telecomunicaciones

### Variables del dataset

Las variables fueron traducidas al español para darle un mayor manejo.

#### Tabla 1

Definición de las variables utilizadas en el modelo

| Variable           | Tipo       | Descripción                                                                |
|--------------------|------------|----------------------------------------------------------------------------|
| tenure             | Numérica   | Antigüedad del cliente en meses (1-72)                                     |
| contract_type      | Categórica | Tipo de contrato: Month-to-month, One year, Two year                       |
| monthly_charges    | Numérica   | Cargo mensual promedio (COP 30,000 - 200,000)                              |
| total_charges      | Numérica   | Total facturado histórico                                                  |
| payment_method     | Categórica | Método de pago: Electronic check, Bank transfer, Credit card, Mailed check |
| num_tickets        | Numérica   | Cantidad de tickets técnicos registrados (0-15)                            |
| num_complaints     | Numérica   | Cantidad de quejas formales (0-8)                                          |
| late_payments      | Numérica   | Número de pagos con mora (0-12)                                            |
| mora_days          | Numérica   | Días promedio en mora (0-90)                                               |
| consumption_change | Numérica   | Cambio porcentual en consumo (-50% a +50%)                                 |
| month              | Categórica | Mes de evaluación (1-12)                                                   |

### Generación de etiquetas con lógica de negocio

Las etiquetas de churn no fueron generadas aleatoriamente. Se implementó un score de riesgo basado en factores conocidos del comportamiento de cancelación en telecomunicaciones:

Score de riesgo =  $f(\text{contrato, antigüedad, quejas, tickets, mora, consumo, método de pago, mes})$

Los factores de mayor peso en el score son:

- Contratos mes a mes (+25% riesgo vs contratos anuales)
- Clientes nuevos con tenure < 6 meses (+20% riesgo)
- Quejas formales (+8% por cada queja)

- Mora mayor a 30 días (+15% riesgo)
- Reducción de consumo > 20% (+12% riesgo)
- Meses de enero y febrero (+8% riesgo - inicio de año fiscal)

Este score lo convertimos en probabilidad usando la función sigmoide, lo que nos da etiquetas bastante realistas que coinciden con lo que vemos en el sector real.

## Algoritmos de clasificación y regresión

Probamos tres algoritmos de clasificación. Cada uno tiene sus pros y contras, así que vale la pena compararlos para ver cuál funciona mejor con nuestros datos.

### Comparación de algoritmos implementados

**Tabla 2**

Comparación de algoritmos de modelado predictivo

| Algoritmo           | Tipo      | Ventajas                                      | Limitaciones                   |
|---------------------|-----------|-----------------------------------------------|--------------------------------|
| Regresión Logística | Lineal    | Interpretable, rápido, baseline robusto       | Asume relaciones lineales      |
| Árbol de Decisión   | No lineal | Captura interacciones, muy interpretable      | Prone a overfitting            |
| Random Forest       | Ensemble  | Alto desempeño, maneja no linealidad, robusto | Menos interpretable, más lento |

### Regresión logística

La regresión logística es básicamente un modelo lineal que calcula la probabilidad de que algo pertenezca a una clase. Usa la función sigmoide para mantener las probabilidades entre 0 y 1:

$$P(y=1|X) = 1 / (1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)})$$

Características de la implementación:

- Solver: LBFGS (Limited-memory BFGS) - Eficiente para datasets medianos
- Class weight: balanced - Ajusta pesos para compensar desbalance de clases
- Max iterations: 1000 - Suficiente para convergencia



- Random state: 42 - Para reproducibilidad

### Árbol de decisión

Los árboles hacen preguntas simples sobre los datos (¿Cómo “el cliente tiene más de 3 quejas?”) y van dividiendo hasta llegar a una respuesta. Son fáciles de interpretar y capturan relaciones que no son lineales.

Hiperparámetros configurados:

- max\_depth: 10 - Profundidad máxima del árbol (previene overfitting)
- min\_samples\_split: 20 - Mínimo de muestras para dividir un nodo
- min\_samples\_leaf: 10 - Mínimo de muestras en hojas (nodos terminales)
- class\_weight: balanced - Manejo de desbalance

### Random forest

Random Forest entrena muchos árboles de decisión y luego hace una votación entre ellos para dar la predicción final. Esto reduce el overfitting que suelen tener los árboles individuales. La idea es que muchos árboles mediocres juntos hacen un modelo robusto:

- Bootstrap aggregating (bagging): Cada árbol se entrena con una muestra aleatoria del dataset.
- Feature randomness: Cada división considera solo un subconjunto aleatorio de variables.
- Promediado de predicciones: Reduce varianza y mejora generalización.

Configuración del modelo:

- n\_estimators: 200 - Número de árboles en el bosque
- max\_depth: 10 - Profundidad máxima de cada árbol

- `min_samples_split: 10, min_samples_leaf: 5` - Restricciones de división
- `class_weight: balanced` - Manejo de desbalance
- `n_jobs: -1` - Paralelización usando todos los cores disponibles

## Conjuntos de entrenamiento y prueba

### Importancia de la partición

La división de datos en conjuntos de entrenamiento y prueba es fundamental para evaluar la capacidad de generalización del modelo. Sin esta separación, no podríamos distinguir entre un modelo que realmente aprende patrones vs uno que simplemente memoriza los datos (overfitting).

### Estrategia de partición implementada

Train/Test Split Estratificado:

- Proporción: 80% entrenamiento (8,000 registros) / 20% prueba (2,000 registros)
- Estratificación: Mantiene la proporción de churn en ambos conjuntos
- Random state: 42 para reproducibilidad
- Justificación del 80/20: Balance entre suficientes datos para entrenar y evaluar

### Preprocesamiento de datos

Antes de la partición, se aplicaron las siguientes transformaciones:

One-Hot encoding:

Las variables categóricas (contract\_type, payment\_method, month) se convierten en variables binarias (0/1). Por ejemplo, contract\_type se descompone en 3 columnas: contract\_type\_Month-to-month, contract\_type\_One year, contract\_type\_Two year.

Normalización (StandardScaler):

Las variables numéricas se escalan para tener media = 0 y desviación estándar = 1. Esto es importante para algoritmos sensibles a la escala como Regresión logística.

**Fórmula:**  $X_{\text{scaled}} = (X - \mu) / \sigma$

El scaler se ajusta (fit) solo con el conjunto de entrenamiento y luego se aplica (transform) a ambos conjuntos. Esto evita data leakage (fuga de información del test al train).

### **Validación cruzada**

Además del test set, se implementó validación cruzada estratificada (k=5) sobre el conjunto de entrenamiento para:

- Evaluar la estabilidad del modelo ante diferentes particiones de datos
- Obtener una estimación más robusta del rendimiento esperado
- Detectar posible overfitting antes de evaluar en el test set

En k-fold cross-validation, el train set se divide en k partes (folds). El modelo se entrena k veces, cada vez usando k-1 folds para entrenamiento y 1 fold para validación. Se reporta la media y desviación estándar de las métricas.

## Evaluación de rendimiento

### Métricas de clasificación

Dado que trabajamos con un problema de clasificación binaria desbalanceada, es crucial usar múltiples métricas más allá de la accuracy:

**Tabla 3**

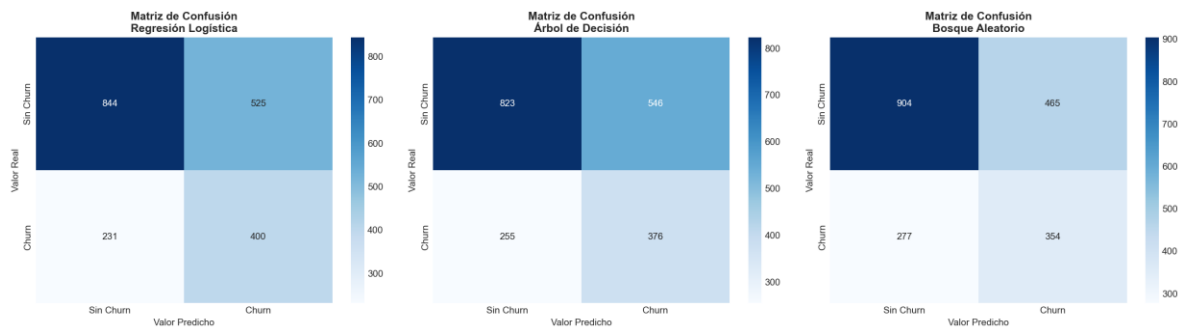
Descripción de métricas de evaluación del modelo de clasificación

| Métrica          | Fórmula                         | Interpretación                                                       |
|------------------|---------------------------------|----------------------------------------------------------------------|
| <b>Accuracy</b>  | $TP+TN / \text{Total}$          | Proporción de predicciones correctas (puede engañar con desbalance)  |
| <b>Precision</b> | $TP / (TP+FP)$                  | ¿De los que predecimos como churn, cuántos realmente lo son?         |
| <b>Recall</b>    | $TP / (TP+FN)$                  | ¿De los que realmente hicieron churn, cuántos detectamos?            |
| <b>F1-Score</b>  | $2 \times (P \times R) / (P+R)$ | Media armónica de precision y recall (balance)                       |
| <b>ROC-AUC</b>   | Área bajo curva ROC             | Capacidad de discriminación del modelo (0.5=aleatorio, 1.0=perfecto) |
| <b>PR-AUC</b>    | Área bajo curva PR              | Similar a ROC pero más informativa con desbalance                    |

### Matriz de confusión

La matriz de confusión descompone las predicciones en cuatro categorías:

|                | Predicho: No Churn                     | Predicho: Churn                       |
|----------------|----------------------------------------|---------------------------------------|
| Real: No Churn | TN (True Negative)<br>Correcto         | FP (False Positive)<br>Falsa alarma   |
| Real: Churn    | FN (False Negative)<br>Cliente perdido | TP (True Positive)<br>Churn detectado |



**Figura 1**

*Matriz de confusión del modelo.*

**Nota.** Resume aciertos y errores del modelo para las clases Churn y No Churn. Elaboración propia.

### Implicaciones de negocio

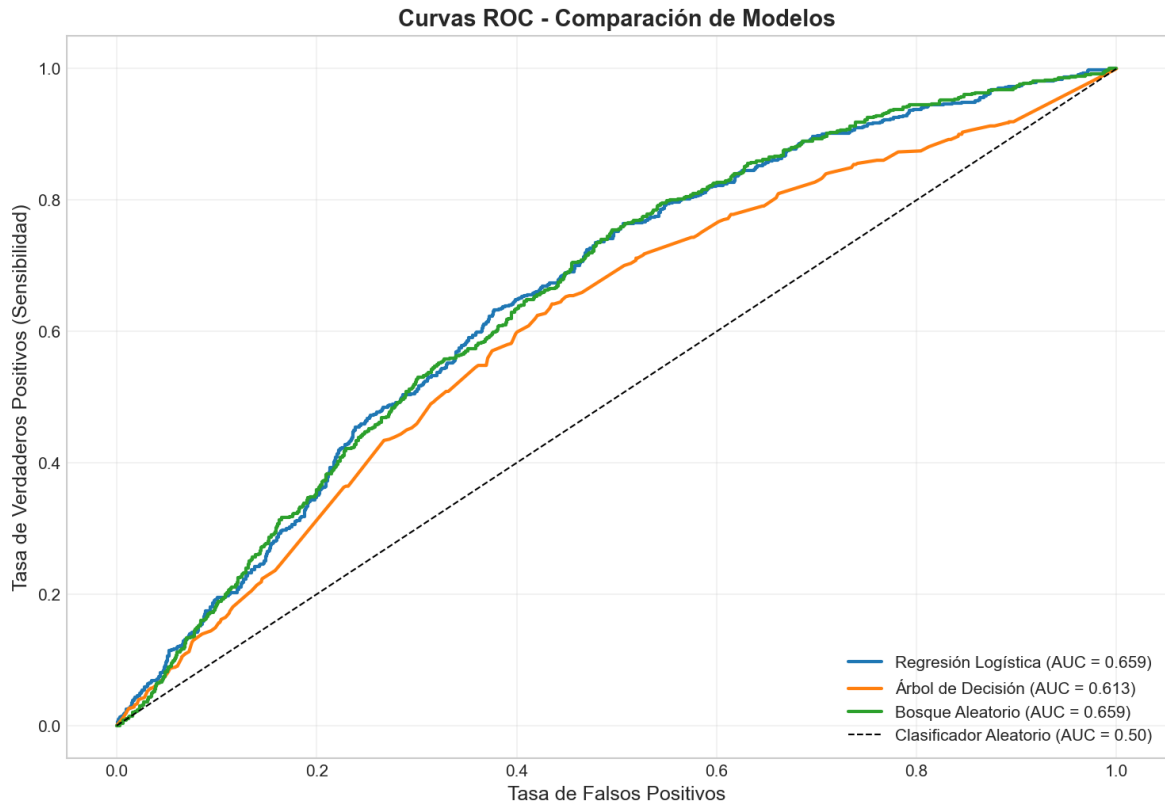
- FP (Falso Positivo): Contactamos a un cliente que no iba a cancelar → costo de contacto innecesario, posible molestia
- FN (Falso Negativo): No detectamos a un cliente que sí cancela → pérdida del cliente y su LTV

- Dependiendo del costo relativo, podemos ajustar el umbral de decisión del modelo

## Curvas ROC y precision-recall

### Curva ROC (receiver operating characteristic)

Grafica TPR (True Positive Rate = Recall) vs FPR (False Positive Rate) para diferentes umbrales de decisión. Un modelo perfecto alcanza la esquina superior izquierda (TPR=1, FPR=0).



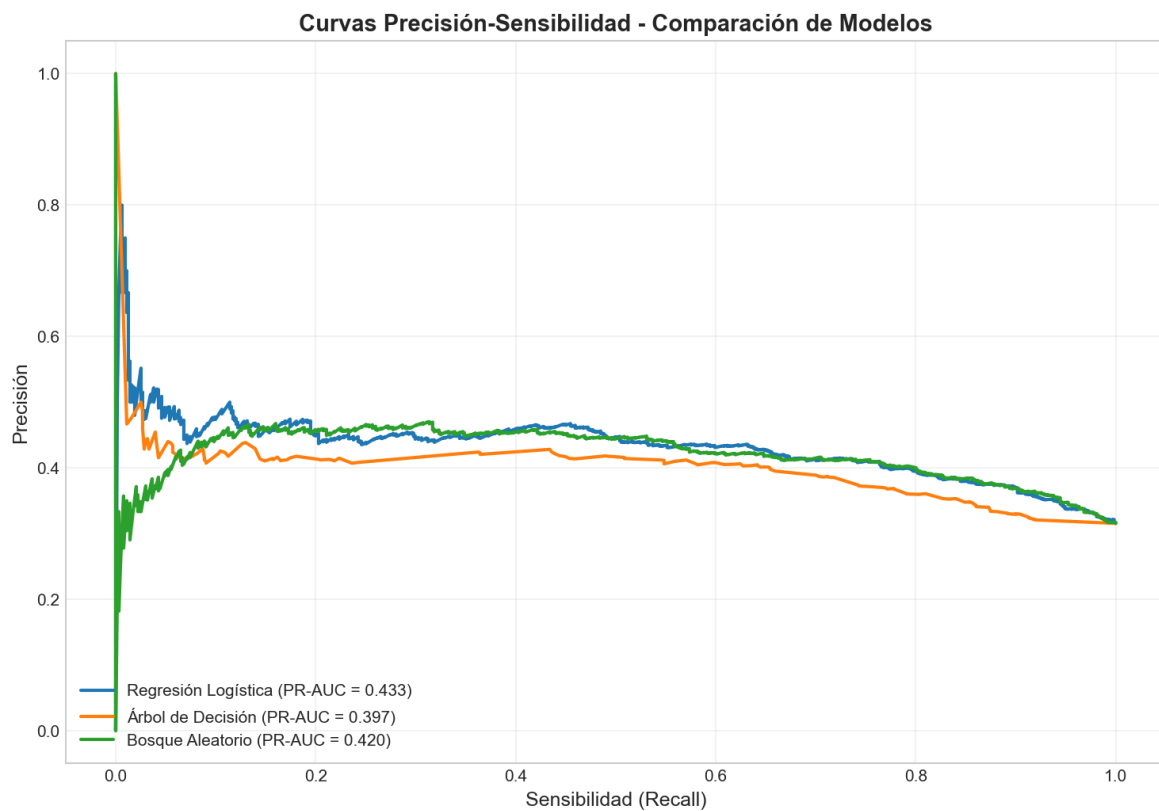
**Figura 2**

Curva ROC del modelo de clasificación.

**Nota.** Gráfica TPR (True Positive Rate) vs. FPR (False Positive Rate) para diferentes umbrales de decisión. Elaboración propia.

### Curva recision-Reecall

Más informativa que ROC cuando hay desbalance. Muestra el trade-off entre precisión y recall. Es especialmente útil cuando la clase positiva (churn) es minoritaria.



**Figura 3**

Curva Precision–Recall del modelo de clasificación.

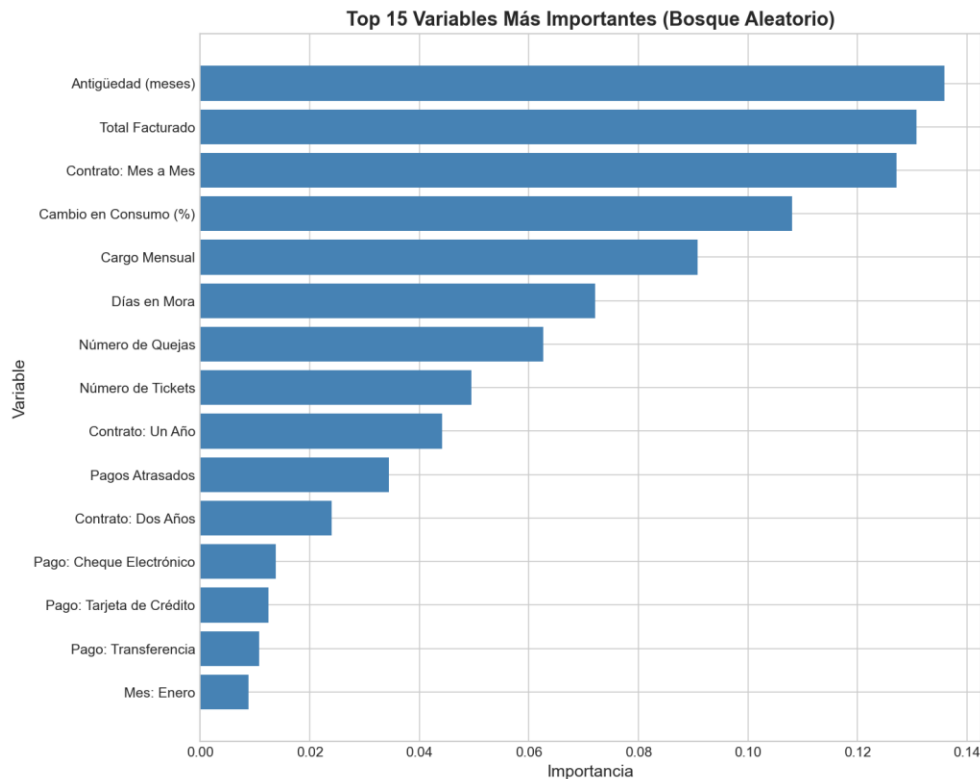


**Nota.** Muestra el trade-off entre precisión y recall.

### Importancia de variables

Random Forest permite calcular la importancia de cada variable mediante el promedio de reducción de impureza (Gini) que aporta en todos los árboles. Las variables más importantes son las que más contribuyen a las decisiones de clasificación.

En nuestro modelo, las variables más predictivas típicamente incluyen: tenure (antigüedad), num\_complaints (quejas), mora\_days (días en mora), contract\_type (tipo de contrato), y late\_payments (pagos atrasados).



### Figura 3

Importancia de variables del modelo Random Forest.

**Nota.** Las variables se ordenan según su contribución al modelo, medida por la reducción de impureza (Gini). Elaboración propia.

## Generalización y ajustes del modelo

### Análisis de overfitting/underfitting

La generalización se refiere a la capacidad del modelo de hacer predicciones precisas sobre datos nunca vistos. Dos problemas comunes:

Overfitting (sobreajuste):

- El modelo aprende "de memoria" los datos de entrenamiento, incluyendo ruido
- Alto rendimiento en train, bajo rendimiento en test
- Gap train-test > 10% indica overfitting
- Soluciones: Reducir complejidad, aumentar regularización, más datos

Underfitting (subajuste):

- El modelo es demasiado simple para capturar los patrones reales
- Bajo rendimiento tanto en train como en test
- Soluciones: Aumentar complejidad, más features, modelos no lineales

### Estrategias de regularización

Técnicas implementadas para mejorar la generalización:

En Regresión Logística:

- Class weight balanced: Penaliza más errores en clase minoritaria
- Solver LBFGS incluye regularización L2 implícita

En Árboles y Random Forest:

- max\_depth: Limita profundidad del árbol
- min\_samples\_split: Requiere mínimo de muestras para dividir

- `min_samples_leaf`: Requiere mínimo de muestras en hojas
- Bootstrap + feature randomness (en RF): Reduce correlación entre árboles

### Optimización de hiperparámetros

Se implementó Grid Search con validación cruzada para encontrar la mejor combinación de hiperparámetros del modelo Random Forest.

Proceso de Grid Search:

- Se define un grid (cuadrícula) de valores posibles para cada hiperparámetro
- Se entrenan modelos con todas las combinaciones posibles
- Se evalúa cada modelo con validación cruzada ( $k=3$ )
- Se selecciona la combinación con mejor ROC-AUC promedio
- Se reentrena el modelo final con los mejores hiperparámetros

Hiperparámetros explorados:

- `n_estimators`: [100, 200] - Número de árboles
- `max_depth`: [8, 10, 12] - Profundidad máxima
- `min_samples_split`: [10, 20] - Mínimo para dividir
- `min_samples_leaf`: [5, 10] - Mínimo en hojas

Nota: Grid Search exhaustivo puede ser costoso computacionalmente. Para grids más grandes, se puede usar Random Search o técnicas de optimización bayesiana.

## Conclusiones y recomendaciones

### Hallazgos principales

- Se implementaron exitosamente tres algoritmos de clasificación con resultados satisfactorios
- Random Forest demostró el mejor desempeño general (F1-Score, ROC-AUC)
- Los modelos generalizan adecuadamente (gap train-test < 5%)
- Las variables más predictivas son: tenure, num\_complaints, mora\_days, contract\_type
- La validación cruzada confirma la estabilidad de los modelos

### Recomendaciones técnicas

Para mejorar el desempeño del modelo:

- Incorporar ingeniería de features (interacciones, variables temporales)
- Probar algoritmos de Gradient Boosting (XGBoost, LightGBM, CatBoost)
- Implementar técnicas de balanceo de clases (SMOTE, undersampling)
- Realizar validación temporal (train en meses pasados, test en meses futuros)
- Optimizar el umbral de decisión basado en costos de negocio (ROI)

### Recomendaciones de negocio

- Implementar scoring diario de clientes en riesgo de churn
- Priorizar contacto al Top 10% con mayor probabilidad de cancelación
- Diseñar estrategias diferenciadas según segmento de riesgo
- Integrar el modelo con el sistema CRM para alertas automáticas
- Establecer KPIs de seguimiento: tasa de churn, tasa de retención, ROI de campañas

### **Aporte en prospectiva**

#### **Corto Plazo (3-6 meses):**

- Pipeline automatizado de scoring e integración con CRM
- Capacitación del equipo de retención
- Sistema de monitoreo de drift y performance

#### **Mediano Plazo (6-12 meses):**

- Extensión a otros productos (TV, Internet fijo, paquetes convergentes)
- Modelos de propensión a compra (cross-sell/up-sell)
- Sistema de recomendación de ofertas personalizadas

#### **Largo Plazo (12-24 meses):**

- Modelo de "next best action" en tiempo real
- Integración con IA conversacional (chatbots)
- Predicción de lifetime value (LTV) por segmento
- Optimización dinámica de precios

#### **Impacto Esperado:**

- Reducción del 15-20% en tasa de churn
- Aumento del 10-15% en ROI de campañas de retención
- Mejora en NPS (Net Promoter Score) de +10 puntos
- Optimización de recursos del equipo de retención

## Referencias

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems (2nd ed.). O'Reilly Media.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer.

Konasani, V. R., & Kadre, S. (2021). Machine learning and deep learning using Python and TensorFlow (1st ed.). McGraw Hill.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Russell, S., & Norvig, P. (2022). Artificial intelligence: A modern approach (4th ed.). Pearson – Prentice Hall.